C Programming Notes & Code Solutions

Mastering Loops, Arrays, and Strings in C

October 11, 2026

Contents

1 Task 1: Countdown Timer (10 to 1) 2
- 1.1 Notes . . . 2
- 1.2 Code . . . 2
- 1.3 Output . . . 2

2 Task 2: Menu-Driven IPL Teams App 2
- 2.1 Notes . . . 2
- 2.2 Code . . . 3

3 Task 3: Guess the Song Game 4
- 3.1 Notes . . . 4
- 3.2 Code . . . 4

4 Task 4: Entry-Controlled vs Exit-Controlled Loops 4
- 4.1 Notes . . . 4
- 4.2 Entry-Controlled (`while`) Code . . . 4
- 4.3 Exit-Controlled (`do-while`) Code . . . 5
- 4.4 Summary . . . 5

1 Task 1: Countdown Timer (10 to 1)

1.1 Notes

- A `for` loop is used when the number of iterations is known in advance.
- Initialize `i = 10`, condition `i >= 1`, and **decrement** with `i--`.
- `\n` prints each number on a new line.

1.2 Code

```c
#include <stdio.h>

int main() {
    for (int i = 10; i >= 1; i--) {
        printf("%d\n", i);
    }
    printf("Liftoff!  \n");
    return 0;
}
```

Listing 1: Countdown Timer in C

1.3 Output

```
10
9
8
...
1
Liftoff!
```

2 Task 2: Menu-Driven IPL Teams App

2.1 Notes

- Use a `while (1)` loop to keep the menu running until the user chooses Exit (`break` exits it).
- Store teams in a **2D character array** (array of strings).
- Use `fgets()` to read strings safely (it also captures spaces in team names like “Royal Challengers Bangalore”).
- Remove the trailing newline (`\n`) that `fgets()` adds.

2.2 Code

```c
#include <stdio.h>
#include <string.h>

int main() {
    char teams[20][50];    // up to 20 teams, each up to 50 chars
    int teamCount = 0;
    int choice;

    // Add 3 favorite teams initially
    strcpy(teams[teamCount++], "Mumbai Indians");
    strcpy(teams[teamCount++], "Chennai Super Kings");
    strcpy(teams[teamCount++], "Royal Challengers Bangalore");

    while (1) {
        printf("\n--- IPL MENU ---\n");
        printf("1. View Favorite Teams\n");
        printf("2. Add a New Team\n");
        printf("3. Exit\n");
        printf("Enter your choice (1-3): ");
        scanf("%d", &choice);
        getchar();    // clear leftover newline from input buffer

        if (choice == 1) {
            printf("Your favorite IPL teams:\n");
            for (int i = 0; i < teamCount; i++) {
                printf("- %s\n", teams[i]);
            }
        }
        else if (choice == 2) {
            if (teamCount >= 20) {
                printf("Team list is full!\n");
                continue;
            }
            printf("Enter new team name: ");
            fgets(teams[teamCount], sizeof(teams[teamCount]), stdin);
            teams[teamCount][strcspn(teams[teamCount], "\n")] = '\0'; //
                remove \n
            printf("%s added successfully!\n", teams[teamCount]);
            teamCount++;
        }
        else if (choice == 3) {
            printf("Goodbye!\n");
            break;
        }
        else {
            printf("Invalid choice, try again.\n");
        }
    }
    return 0;
}
```

Listing 2: IPL Teams Menu Application

3 Task 3: Guess the Song Game

3.1 Notes

- C has no built-in do-while-friendly "random pick" — use `rand()` with `srand(time(NULL))` to seed it (so the song changes each run).
- `rand() % 3` gives a number 0–2, used as the array index.
- Use a **do-while loop** (exit-controlled): the user guesses at least once, and the loop repeats `while (strcmp(guess, song) != 0)`.
- `strcmp()` returns 0 when two strings are equal.

3.2 Code

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

int main() {
    char songs[3][50] = {"Blinding Lights", "Kesariya", "Shape of You"};
    char guess[50];

    srand(time(NULL));                      // seed random generator
    int index = rand() % 3;                   // pick 0, 1, or 2
    char *song = songs[index];

    printf(" Guess the Song! (Hint: it's a popular hit)\n");

    do {
        printf("Enter your guess: ");
        fgets(guess, sizeof(guess), stdin);
        guess[strcspn(guess, "\n")] = '\0'; // remove trailing newline

        if (strcmp(guess, song) != 0) {
            printf(" Wrong! Try again.\n");
        }
    } while (strcmp(guess, song) != 0);       // repeat until correct

    printf(" Correct! The song was: %s\n", song);
    return 0;
}
```

Listing 3: Guess the Song Game

4 Task 4: Entry-Controlled vs Exit-Controlled Loops

4.1 Notes

4.2 Entry-Controlled (`while`) Code

Feature	Entry-Controlled (`for` / `while`)	Exit-Controlled (`do-while`)
Condition checked	**Before** the body runs	**After** the body runs
Minimum executions	0 times	At least 1 time
Best used when	Work may not be needed at all	Work must happen at least once

Table 1: Comparison of Loop Types in C

```c
#include <stdio.h>

int main() {
    int count = 10;

    while (count < 5) {          // condition is FALSE at the start
        printf("This will NEVER print\n");
        count++;
    }
    printf("Loop skipped - body executed 0 times\n");
    return 0;
}
```

Listing 4: Entry-Controlled Loop Example

Output: `Loop skipped - body executed 0 times`
Because the condition `count < 5` is false at the start, the body **never executes**.

4.3 Exit-Controlled (`do-while`) Code

```c
#include <stdio.h>

int main() {
    int count = 10;

    do {
        printf("This prints ONCE even though condition is false\n");
        count++;
    } while (count < 5);         // condition false, but body already ran
       once

    printf("Body executed exactly 1 time\n");
    return 0;
}
```

Listing 5: Exit-Controlled Loop Example

Output:

```
This prints ONCE even though condition is false
Body executed exactly 1 time
```

Because `do-while` checks the condition **after** running the body, the body executes **at least once** — even when the condition is false from the beginning.

4.4 Summary

- Use **entry-controlled** loops (`for`/`while`) when repetition might not be needed at all.

- Use **exit-controlled** loops (`do-while`) when the task must run at least once — like the menu (Task 2) and song-guessing game (Task 3), where a do-while is the natural choice.